

HASHDIT

HashDit

Smart Contract Security

Venus Liquidity Hub Capacity Lens

Security Audit Report

Venus

22 Sep 2026 - 24 Sep 2026

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Period	22 Sep 2026 - 24 Sep 2026
Overall Risk	Low
Project Name	Venus Liquidity Hub Capacity Lens Audit
Github	https://github.com/VenusProtocol/venus-liquidity-hub/pull/20
Commit	6715424bd06c93ab6f63170ed407431d64374845

Smart Contract List

No.	Contract Name	Verdict	Details
1	HubLens.sol	Low	[L01]-Fixed
2	AdapterCoreV1.sol	Low	[L01]-Fixed
3	AdapterFRV.sol	Informational	[I02]-Ack
4	Hub.sol	Informational	[I01]-Ack
5	HubCapacityMath.sol	Passed	-

Findings

[L01] HubLens over-reports deposit capacity for interest-accruing YieldGroups at their cap	
Contract	contracts/lens/HubLens.sol; contracts/adapters/AdapterCoreV1.sol
Severity	Low

Description	HubLens.maxDeposit (and maxMint, depositCapacityBreakdown) price each YieldGroup's Hub cap against the group's current totalAssets(), read through a pure view. For a Venus Core position the adapter values it off exchangeRateStored() -- the exchange rate as of the last accrual, which a view cannot advance:
	
	JavaScript
	`// AdapterCoreV1.totalAssets`
	`uint256 gross = (vBal *`
	`IVToken(resource).exchangeRateStored()) / EXP_SCALE;`
	
	A real deposit, however, settles pending interest BEFORE it prices caps: deposit -> _accrueFees() -> _accrueResources() -> YieldGroup.accrue() -> vToken.accrueInterest(), which raises the stored rate to the current block. _routeDeposit / _capRoomForDeposit then read the raised balance:
	
	JavaScript
	`// AdapterCoreV1.accrue -- run by the deposit path before caps are priced`
	`function accrue(address resource) external override {`
	`IVToken(resource).accrueInterest(); //`
	`exchangeRateStored() is now fresh to the block`
	`}`
	
	For a group already at its Hub cap, room = cap - balance, so the settled (higher) balance leaves less room than the lens computed. The lens therefore over-reports by the pending interest and deposit(HubLens.maxDeposit(hub)) reverts -- contradicting the interface's explicit guarantee:
	
	JavaScript
	`// IHubLens.maxDeposit`
	`/// Never overstated: hub.deposit(maxDeposit(hub)) fits.`
	
	Impact

	For a cap-bound Core / FRV group, deposit(HubLens.maxDeposit(hub)) reverts with HubCapacityExceeded; the over-report equals the pending interest. It is fail-safe -- an atomic revert, never a fund loss or an over-cap deposit -- but it breaks a documented invariant that integrators are told to rely on, on the primary asset family, under normal (non-adversarial) conditions. For example, a group at an absolute cap of 1000 whose mark is 900 reports 100 of room; the deposit settles interest, raising the mark to 905, so only 95 of room remains and deposit(100) reverts HubCapacityExceeded(100, 95). maxMint and depositCapacityBreakdown inherit the same over-report, as all three read the stale totalAssets() via _readDepositQueue.

Recommendation	Value the Hub cap in the lens against an interest-current mark rather than the stale exchangeRateStored(), or trim a small conservative margin on the Hub-cap dimension -- the venue dimension already does exactly this for the same staleness:
	
	JavaScript
	`// AdapterCoreV1.maxDeposit -- margin applied on the venue dimension only`
	`return raw - (raw / 1000);`
	
	Alternatively, weaken the IHubLens wording from 'Never overstated' to 'holds for the current, static state' and document the read-vs-execute drift so integrators size with a buffer. Add a regression test with an interest-accruing YieldGroup (and ideally a fork test that advances a block between the maxDeposit read and the deposit) to guard the invariant.

Status	Fixed in PR #29 (reviewed up to 11f1028; merged in 43275cb), primarily in commit 162d2ce. HubLens now trims an accrual margin off each YieldGroup's room before reporting capacity, so the interest a deposit settles before pricing caps no longer causes deposit(maxDeposit) to revert. The trimmed figure is re-verified against the cascade's room, and the same treatment is applied to maxMint and depositCapacityBreakdown. IHubLens no longer claims the figure is "never overstated" and instead documents that an accrual beyond the margin reverts. During the review, we noted that the initial margin (0.1% of the group's whole balance) could exceed a large group's remaining room and make HubLens report zero capacity. The team reduced it to 0.01% in 92b8043 and documented in the code that it is sized on the balance rather than the room.

```JavaScript
// HubLens.sol — maxDeposit
(assets,) = HubCapacityMath.trimmedCapacity(groups,
hubTotalAssets, ACCRUAL_MARGIN_DIVISOR);   // 10_000 = 0.01%
```

[I01] Hub.maxDeposit / maxMint return type(uint256).max, including while paused	
Contract	contracts/Hub/Hub.sol
Severity	Informational

Description	The maxDeposit / maxMint overrides were removed; both now inherit OpenZeppelin's implementations and return type(uint256).max, even while the Hub is paused (the previous override returned 0 when paused):
	
	JavaScript
	`// Hub.sol -- overrides deliberately removed`
	`// maxDeposit / maxMint inherit OZ and return type(uint256).max`
	`// (even while paused; the old override returned 0).`
	`// Caps are still enforced by _routeDeposit, which reverts HubCapacityExceeded.`
	
	Caps remain enforced on-chain by _routeDeposit, and deposits revert while paused via the whenNotHubPaused modifier, so there is no fund risk. The real capacity figure now lives on HubLens.
	
	Impact
	A generic ERC-4626 consumer (aggregator, zap, keeper) that sizes a deposit off Hub.maxDeposit now sees unlimited capacity — including during a pause — and its deposit reverts. It must instead read HubLens.maxDeposit (which returns 0 while paused). This violates ERC-4626's clause that maxDeposit MUST return the maximum that would not revert; it is a deliberate, README-documented deviation. No fund risk; the impact is confined to off-chain integrators that size deposits off this view.

Recommendation	Document prominently that all off-chain capacity reads must use HubLens, not Hub.maxDeposit. Ensure HubLens is deployed before (or atomically with) the HubBeacon.upgradeTo that flips every Hub's maxDeposit to type(uint256).max, and verify that any integrators/keepers/zaps do not size deposits off Hub.maxDeposit.
Status	Acknowledged. Intentional: Hub.maxDeposit/maxMint keep OZ's type(uint256).max to stay off the hot path and save bytecode; the real

	figure (0 while paused) is on HubLens. No fund risk — paused deposits revert via whenNotHubPaused, and caps are enforced by _routeDeposit. Since the Hub implementation changes in PR #20 and PR #29(6c18197), deploying them requires a new HubImpl and a beacon upgrade; HubLens should be deployed and integrators migrated to it before that upgrade, as Hub.maxDeposit will then return type(uint256).max.

[I02] AdapterFRV.spotAPYBps distrusts fixedAPY but trusts reserveFactor, underflowing on a misreporting vault	
Contract	contracts/adapters/AdapterFRV.sol
Severity	Informational

Description	spotAPYBps reports the FRV supplier's net rate. It defends against a misreporting fixedAPY by saturating it before the multiply, but trusts that reserveFactor <= MANTISSA_ONE (1e18):
	
	JavaScript
	`uint256 grossApy = cfg.fixedAPY > type(uint64).max ? type(uint64).max : cfg.fixedAPY;`
	`return uint64(grossApy - (grossApy * cfg.reserveFactor) / MANTISSA_ONE);`
	
	If the same class of misreporting vault returns reserveFactor > 1e18, the subtraction underflows and the call reverts. The identical trust already governs the valuation path _accruedTotalAssets:
	
	JavaScript
	`// AdapterFRV._accruedTotalAssets`
	`uint256 netInterest = totalInterest - (totalInterest * cfg.reserveFactor) / MANTISSA_ONE;`
	
	Impact
	spotAPYBps is an informational view, and HubLens.spotAPYBps wraps the per-group call in try/catch, so an underflow only drops that group from the blended APY -- no cascade failure, no fund risk; severity is bounded by this containment. Note that _accruedTotalAssets (a valuation-path function, not wrapped in try/catch) carries the same reserveFactor assumption and additionally does not saturate fixedAPY, so it is the more dangerous surface (a misreporting vault would revert totalAssets() and halt the Hub); that behaviour is pre-existing and outside this PR's diff.

Recommendation	Enforce reserveFactor <= MANTISSA_ONE at its source (the FRV vault controller, at config time). For defense-in-depth consistent with the existing fixedAPY saturation, clamp rather than revert so the view degrades gracefully instead of reverting:
	
	JavaScript
	`uint256 rf = cfg.reserveFactor > MANTISSA_ONE ? MANTISSA_ONE : cfg.reserveFactor;`
	`return uint64(grossApy - (grossApy * rf) / MANTISSA_ONE);`
	
	Apply the same clamp (and a fixedAPY saturation) to _accruedTotalAssets. A reverting range check does not help -- it merely replaces the underflow panic with another revert of the same view.

Status	Acknowledged. Reasoning verified: the reserveFactor <= 1e18 / fixedAPY ∈ (0, 10_000] bounds are enforced at vault creation in InstitutionalVaultController._validateVaultConfig (fixed-rate-vaults, bscmainnet 0x6D9e91cB766259af42619c14c994E694E57e6E85); config is write-once, vaults enter only via ACM-gated addResource, and a broken bound makes _accruedTotalAssets revert (Hub fails closed) rather than misprice.
	
	JavaScript
	`// InstitutionalVaultController.sol – _validateVaultConfig`
	`if (vaultConfig.fixedAPY == 0 || vaultConfig.fixedAPY > MAX_APY_BPS) revert InvalidConfig();`
	`if (vaultConfig.reserveFactor > MANTISSA_ONE) revert InvalidConfig();`